A Final Project Report On

Cloud-Based IoT Air Quality Monitoring System



Submitted in partial fulfillment of the
Requirements for the Degree of Bachelor of Computer Engineering
Awarded by Pokhara University

Submitted By:

Anish Tharu (23070332)

Kushal Chaudhary (23070346)

Ronit Pradhan (23070357)

Sashank Rimal (23070366)

Supervisor:
Er. Santosh Poudel
Assistant Professor

School of Engineering
Faculty of Science and Technology
POKHARA UNIVERSITY

Aug 2026

COPYRIGHT

The author has granted authorization to the Pokhara University Central Library and Library of School of Engineering to grant unrestricted access to this report for inspection. Furthermore, the author has agreed to the potential for extensive reproduction of this report for educational purposes by either the supervisors or, in their absence, by the program coordinator wherein the project was done. It is understood that proper acknowledgment will be made by both the author of the report and the School of Engineering, Pokhara University in any utilization of the report's contents. Unauthorized copying, publication, or any other form of exploitation of this report for financial gain without explicit consent from the School of Engineering, Pokhara University and the author is strictly prohibited. Requests seeking permission to replicate or employ any segment of the report's content should be directed to:

Program Coordinator
School of Engineering
Pokhara University

STUDENT'S DECLARATION

We hereby declare that this project work entitled **Cloud-Based IoT Air Quality Monitoring System** is based on our original work. All concepts, data, code, and any other work from external sources have been properly cited and referenced in accordance with the guidelines provided by School of Engineering, Pokhara University

We owe all the liabilities relating to the authenticity and originality of this project work and project report.

1. Anish Tharu (23070332) …….
2. Kushal Chaudhary (23070346) …….
3. Ronit Pradhan (23070357) …….
4. Sashank Rimal (23070366) …….

Date: ……

SUPERVISOR'S RECOMMENDATION

This is to certify that this project report entitled **Cloud-Based IoT Air Quality Monitoring System** prepared and submitted by below listed team of students in partial fulfilment of the requirements of the degree of Bachelor of Computer Engineering awarded by Pokhara University, has been prepared and completed under my supervision.

I hereby recommend the same for acceptance by School of Engineering, Pokhara University.

1. Anish Tharu (23070332)
2. Kushal Chaudhary (23070346)
3. Ronit Pradhan (23070357)
4. Sashank Rimal (23070366)

……

Er. Santosh Poudel

Assistant Professor

School of Engineering

Pokhara University

Date: ……



School of Engineering
POKHARA UNIVERSITY

EXTERNAL EXAMINER'S RECOMMENDATION

The undersigned certified that they have evaluated this project report entitled **Cloud-Based IoT Air Quality Monitoring System** submitted by **Anish Tharu, Kushal Chaudhary, Ronit Pradhan, Sashank Rimal** and their oral presentation for partial fulfillment of the degree of Bachelor of Computer Engineering and recommended to the School of Engineering, Pokhara University for acceptance of this project work/report.

…….



School of Engineering
POKHARA UNIVERSITY

LETTER OF APPROVAL

This project report entitled **Cloud-Based IoT Air Quality Monitoring System** submitted by **Anish Tharu, Kushal Chaudhary, Ronit Pradhan, Sashank Rimal** for partial fulfillment of the degree of Bachelor of Computer Engineering has been accepted by the School of Engineering, Pokhara University upon the recommendations of Supervisor and with the approval by the following examiner.

…….

…….

Er. Santosh Poudel
Assistant Professor
School of Engineering
Pokhara university

Date: …….

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our project titled **"Cloud-Based IoT Air Quality Monitoring System."** The accomplishment of this work would not have been possible without the encouragement, guidance, and assistance of several individuals and institutions who contributed directly and indirectly to this research.

We extend our heartfelt appreciation to the **School of Engineering, Pokhara University**, and the **Department of Computer Engineering** for providing the academic environment, technical resources, and continuous encouragement necessary to carry out this project effectively. The institutional support and structured academic framework played a vital role in the successful development and implementation of this system.

We are especially grateful to our respected supervisor, **Er. Santosh Poudel**, whose expert guidance, constructive feedback, and constant support played a crucial role in shaping the direction and quality of this work. Their mentorship motivated us to remain disciplined and focused throughout the development process.

Finally, we express our deepest gratitude to our friends and family for their unwavering encouragement and moral support throughout this journey.

Anish Tharu (23070332)

Kushal Chaudhary (23070346)

Ronit Pradhan (23070357)

Sashank Rimal (23070366)

Date: ……

ABSTRACT

Air pollution and poor indoor air quality can negatively affect human health and environmental safety. This project presents an IoT Based Air Quality Monitoring System that was designed, built, and tested to monitor harmful gas concentrations and environmental conditions in real time. The system uses an ESP32 microcontroller with MQ2 and MQ135 gas sensors to detect smoke, combustible gases, and general air pollutants, alongside a DHT11 sensor to measure temperature and humidity. Collected data is displayed locally on a 16×2 LCD and simultaneously logged to Google Sheets, from where a Python backend serves the data to a custom web dashboard, deployed on Vercel, for remote real-time and historical monitoring.

The system continuously analyzes air quality using a weighted scoring model that classifies conditions into Good, Moderate, and Poor levels, communicated consistently across the local LCD and the web dashboard. The completed prototype was tested and successfully demonstrated real-time sensor readings, live dashboard visualization, and historical data logging with search, filtering, and CSV export. The system proved to be low-cost, functional, and suitable for smart environmental monitoring applications in homes, laboratories, and small workspaces.

Keywords: *Air Pollution, ESP32, Gas Detection, MQ2 Sensor, MQ135 Sensor, DHT11 Sensor, LCD Display, Cloud Dashboard*

TABLE OF CONTENTS

ABSTRACT i

TABLE OF CONTENTS ii

LIST OF FIGURES iv

LIST OF TABLES v

ABBREVIATIONS vi

CHAPTER 1 1

INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Objective 3

1.4 Applications 3

1.5 Project Features 4

1.6 Feasibility Analysis 5

1.6.1 Economic Feasibility 5

1.6.2 Technical Feasibility 5

1.6.3 Operational Feasibility 5

1.7 System Requirement 6

1.7.1 Hardware Requirement 6

1.7.2 Software Requirement 8

CHAPTER 2 9

LITERATURE REVIEW 9

2.1 Evolution from Localized Systems to IoT Architecture 9

2.2 Wireless Protocols and the Adoption of ESP32 9

2.3 Cloud Visualization Platforms: 9

2.4 Automated Fail-Safe Alerting Mechanisms 10
2.5 Advanced Trends in Smart Environmental Nodes 10
CHAPTER 3 11
METHODOLOGY 11
3.1 Software Development Life Cycle (SDLC) 11
3.2 Use Case Diagram 12
3.3 System Design and Architecture 13
3.4 Circuit Implementation 13
CHAPTER 4 14
EPILOGUE 14
4.1 Expected Output 14
4.2 Budget Analysis 14
4.3 Work Schedule 15
REFERENCES 16

LIST OF FIGURES

Figure 1.1: ESP32 Board 6

Figure 1.2: MQ2 Gas Sensor 6

Figure 1.3: DHT11 Sensor 6

Figure 1.4: 16x2 LCD 7

Figure 1.5: Breadboard 7

Figure 1.6: Jumper Wire 7

Figure 3.1: Waterfall Model (SDLC) 11

Figure 3.2: Use Case Diagram 12

Figure 3.3: Block diagram 13

Figure 3.4: Circuit Diagram 13

Figure 4.1: Gantt Chart 15

LIST OF TABLES

Table 1.1: Software Requirement 8

Table 4.1: Budget Analysis 14

ABBREVIATIONS

ADC	Analog to Digital Converter
API	Application Programming Interface
CSS	Cascading Style Sheets
CSV	Comma-separated Values
DHT	Digital Humidity and Temperature
GSM	Global System for Mobile Communication
HTML	Hyper Text Markup Language
IAQ	Indoor Air Quality
IDE	Integrated Development Environment
IoT	Internet of Things
LCD	Liquid Crystal Display
LPG	Liquefied Petroleum Gas
MOS	Metal Oxide Semiconductor
SQL	Structured Query Language
Wi-Fi	Wireless Fidelity

CHAPTER 1
INTRODUCTION

1.1 Background

Air quality monitoring has become increasingly important with the growth of urbanization, industrial activities, and the use of enclosed indoor spaces. People spend a significant amount of their time indoors in environments such as homes, classrooms, laboratories, offices, and workshops. Therefore, monitoring environmental conditions such as temperature, humidity, smoke, and the presence of certain gases can contribute to improved awareness of indoor environmental conditions.

Poor indoor air quality can be associated with discomfort and potential health and safety concerns. Temperature and relative humidity are important environmental parameters because they influence thermal comfort and the overall indoor environment. Similarly, smoke and combustible gases may present safety risks when they accumulate in enclosed spaces. Conventional monitoring devices often provide information only at the location where the device is installed. This limits the ability of users to monitor conditions remotely.

The Internet of Things (IoT) provides a way to connect physical sensing devices to networked services so that collected information can be accessed remotely. Low-cost microcontrollers with integrated wireless connectivity, such as the ESP32, make it possible to develop compact monitoring systems with sensor data acquisition, local processing, and Internet connectivity [8], [19], [22].

This project presents a low-cost IoT-based air quality monitoring system using an ESP32 microcontroller. The system integrates MQ2 and MQ135 gas sensors for detecting changes in gas and smoke sensor responses and a DHT11 sensor for measuring temperature and humidity. The collected readings are displayed locally using a 16×2 LCD and transmitted through Wi-Fi for cloud-based logging.

The system uses Google Sheets as a cloud-based data store. A Python backend retrieves the logged data and provides it to a custom web dashboard deployed using Vercel. The dashboard provides live sensor information and historical records, allowing users to monitor the system remotely through a web browser.

In addition to displaying individual sensor readings, the system combines the MQ2 and MQ135 sensor responses using a project-specific weighted scoring method. The resulting Air Quality Score is used to classify the current condition into three categories: Good, Moderate, and Poor. The classification is presented through the system interfaces to provide users with an easily understandable indication of the monitored condition.

The proposed system is intended as a low-cost prototype for indoor and small-scale environmental monitoring. It is not intended to replace certified industrial gas detection equipment or standardized air-quality monitoring stations.

1.2 Problem Statement

Air quality and environmental conditions can change over time because of smoke, combustible gases, inadequate ventilation, temperature changes, and other environmental factors. In many small indoor environments, users may not have access to continuous monitoring equipment because commercial monitoring systems can be expensive or may provide limited remote-access functionality.

Traditional localized monitoring devices generally provide information through a display, indicator, or alarm at the physical installation site. Consequently, a user who is away from the monitored location may not be able to determine the current environmental condition without physically visiting the location.

There is therefore a need for a low-cost monitoring system that can:

- continuously acquire environmental and gas-sensor readings
- display the readings locally
- transmit sensor data through a wireless network
- store historical readings for later analysis
- provide remote access through a web-based interface
- present sensor information in an understandable form.

The proposed system addresses these requirements by combining low-cost sensors, an ESP32 microcontroller, Wi-Fi connectivity, cloud-based data logging, a Python backend, and a custom web dashboard.

1.3 Objective

The main objectives of the project are:

- To develop a low-cost IoT-based system using ESP32, MQ2, MQ135, and DHT11 sensors for monitoring air-quality-related sensor readings, temperature, and humidity, with local display and wireless data transmission.
- To develop a cloud-connected web dashboard for real-time and historical visualization of sensor data, including air-quality classification, data filtering, and CSV export.

1.4 Applications

The developed prototype can be applied to small-scale environments where continuous environmental monitoring is useful. Potential applications include:

1.4.1 Residential Environments

The system can be used in homes and kitchens to monitor changes in smoke and combustible-gas sensor responses. Temperature and humidity readings can also provide information about indoor environmental conditions.

1.4.2 Academic and Laboratory Environments

The system can be used in classrooms, laboratories, and small workshops as an educational and prototype monitoring solution. It can demonstrate the use of IoT technologies for environmental data acquisition and remote visualization.

1.4.3 Small Workspaces

Small offices, workshops, and enclosed workspaces can use the system to observe changes in environmental conditions and sensor readings over time.

1.4.4 Remote Monitoring of Small Properties

Because the system provides a web-based dashboard, sensor data can be accessed remotely when the monitoring device and network connection are operational.

The proposed system should be considered a prototype monitoring solution rather than a certified safety or industrial gas-detection system.

1.5 Project Features

The major features of the proposed system are described below.

1.5.1 Multi-Sensor Data Acquisition

The system uses MQ2 and MQ135 sensors to obtain gas and smoke-related sensor responses. A DHT11 sensor is used to obtain temperature and humidity readings.

1.5.2 Local Data Display

A 16×2 LCD is used to display sensor information locally. This allows users near the device to observe the current readings without requiring access to the web dashboard.

1.5.3 Wireless Data Transmission

The ESP32 provides built-in Wi-Fi connectivity and is responsible for transmitting the collected data through the network.

1.5.4 Cloud-Based Data Logging

Sensor readings are logged to Google Sheets. The stored readings provide a historical record that can be retrieved by the backend and presented through the dashboard.

1.5.5 Web-Based Monitoring

A custom web dashboard provides access to current and historical sensor information through a web browser. The dashboard is implemented using HTML, CSS, JavaScript, and Chart.js, with a Python backend handling data retrieval.

1.5.6 Air Quality Classification

The system combines normalized MQ2 and MQ135 readings into a project-specific Pollution Index and converts the resulting value into an Air Quality Score. The score is categorized as Good, Moderate, or Poor.

The Air Quality Score is a project-specific indicator and should not be interpreted as an official standardized Air Quality Index (AQI).

1.5.7 Historical Data and CSV Export

The dashboard provides a historical table of sensor readings. Users can filter the records and export the available data as a CSV file for offline analysis or record keeping.

1.4 Feasibility Analysis

A feasibility analysis was conducted to determine whether the proposed system could be developed using the available hardware, software, financial resources, and technical skills.

1.6.1 Technical Feasibility

The system is technically feasible because the ESP32 provides integrated Wi-Fi connectivity and sufficient interfaces for connecting the required sensors and display. MQ2 and MQ135 provide analog sensor outputs, while the DHT11 provides digital temperature and humidity readings.

The ESP32 can acquire sensor readings, perform basic processing, control the LCD, and communicate through Wi-Fi. The software components used in the project, including Arduino IDE, Python, HTML, CSS, JavaScript, and Chart.js, provide the required development capabilities. For cloud-based monitoring, Google Sheets is used to store the collected readings, while a Python backend retrieves the data and provides it to the web dashboard.

1.6.2 Economic Feasibility

The proposed system is economically feasible because it uses commonly available low-cost components such as the ESP32, MQ2, MQ135, DHT11, LCD, breadboard, jumper wires, batteries, and a buck converter.

The software development environment and major software components used for the prototype do not require expensive proprietary development tools. The system can therefore be developed using commonly available hardware and software resources.

1.6.3 Operational Feasibility

The system is designed to operate with minimal manual interaction after the hardware has been configured and connected to a Wi-Fi network.

The LCD provides local access to sensor information, while the web dashboard provides remote access to the collected data.The use of a browser-based dashboard eliminates the need for a dedicated mobile application and allows users to access the monitoring interface from supported web browsers.

1.7 System Requirement

The system requires both hardware and software components for sensing, processing, communication, data storage, and visualization

1.7.1 Hardware Requirements

The major hardware components are:

ESP32 Board:

The ESP32 acts as the main controller. It reads the connected sensors, processes the sensor readings, controls the LCD, and provides Wi-Fi connectivity [8].

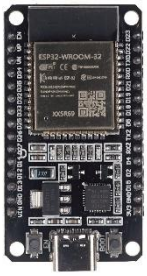


Figure 1.1: ESP32 Board

MQ2 Gas Sensor:

The MQ2 sensor is used to detect changes in the presence of smoke and combustible gases. The sensor produces an electrical response that can be read by the microcontroller [13].



Figure 1.2: MQ2 Gas Sensor

MQ135 Gas Sensor:

The MQ135 is a broad-response gas sensor used for detecting changes associated with several gases and air contaminants [14].



Figure1.3: MQ135 Sensor

DHT11 Temperature and Humidity Sensor:

The DHT11 is used to measure ambient temperature and relative humidity [15].

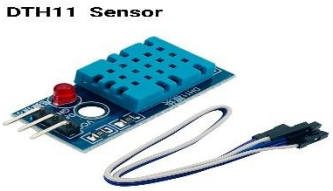


Figure 1.4: DHT11 Sensor

16×2 LCD:

The LCD provides local display of sensor readings and system information.



Figure 1.5: 16×2 LCD

Breadboard:

The breadboard is used for assembling and testing the circuit connections..

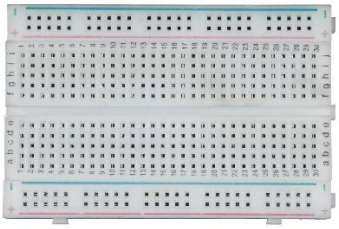


Figure 1.7: Breadboard

Jumper wire:

Jumper wires are used to establish electrical connections between the ESP32 and other components.



Figure 1.8: Jumper Wire

Buck Converter:

The buck converter is used to regulate the battery supply voltage to a suitable level for the system.



Figure 1.9: Buck Converter

Rechargeable Li-ion Batteries:

Two rechargeable batteries are used as the portable power source for the prototype.



Figure 1.10: Rechargeable Batteries

1.4.1 Software Requirements

The software components used in the project are summarized below.

Table 1.1: Programming language

S.N.	Programming language	Purpose
1	Embedded C/C++	ESP32 firmware development and sensor integration
2	Python	Backend processing and retrieval of data from Google Sheets
3	HTML	Structure of the web dashboard
4	CSS	Styling and layout of the dashboard
5	JavaScript	Dashboard interaction and data handling

Table 1.2: Cloud Platform & Services

S.N.	Platform/Services	Purpose
1	Google Sheets API	Cloud-based storage/logging of sensor readings
2	Vercel	Deployment and hosting of the web dashboard/backend.

Table 1.3: Libraries

S.N.	Library	Purpose
1	WiFi.h	Provides Wi-Fi connectivity for ESP32
2	WiFiManager.h	Provides Wi-Fi configuration functionality
3	WebServer.h	Supports web-server functionality used during Wi-Fi configuration
4	DHT.h	Reads temperature and humidity from DHT11
5	LiquidCrystal_I2C.h	Controls the I²C-based 16×2 LCD.
6	Chart.js	Generates charts for sensor data visualization

Table 1.4: Development Tools

S.N.	Tool	Purpose
1	Arduino IDE	Firmware development, compilation, and uploading
2	Python environment	Backend development
3	Web browser	Accessing and testing the dashboard
4	Windows OS	Supported Arduino IDE and development tools during development.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The Internet of Things (IoT) enables physical devices equipped with sensors to collect and exchange data through communication networks. IoT-based environmental monitoring systems combine sensors, microcontrollers, communication technologies, and data visualization platforms to monitor environmental conditions remotely. Several studies have explored the use of low-cost sensors and microcontrollers for air-quality monitoring. This chapter reviews selected technologies and related works relevant to the proposed Cloud-Based IoT Air Quality Monitoring System.

2.2 IoT-Based Air Quality Monitoring

IoT technology provides an effective approach for monitoring environmental conditions because sensor readings can be collected continuously and transmitted to remote platforms. Compared with conventional standalone monitoring devices, IoT-based systems can provide remote access to sensor information and allow historical data to be stored for later analysis.

Previous studies have demonstrated the use of microcontrollers, gas sensors, and wireless communication for air-quality monitoring. These systems show that low-cost IoT devices can be used to collect environmental information and present it to users through digital interfaces [19], [22].

The proposed system follows this approach by using an ESP32 to collect sensor readings and transmit the data through Wi-Fi. The collected information is stored in a cloud-based service and displayed through a web dashboard.

2.3 ESP32 and Sensors

The ESP32 is a low-cost microcontroller with integrated Wi-Fi and Bluetooth capabilities. Its wireless connectivity and available input/output interfaces make it suitable for IoT applications [8]. In the proposed system, the ESP32 acts as the main controller for acquiring sensor readings, processing the data, and transmitting the information to the cloud.

The MQ2 sensor is commonly used for detecting smoke and combustible gases [13], while the MQ135 is a broad-response gas sensor commonly used in air-quality-related applications [14]. These sensors are used in the proposed system to obtain gas and smoke-related sensor responses.

The DHT11 sensor is used to measure temperature and relative humidity [15]. These environmental measurements are included with the gas-sensor readings to provide additional information about the monitored environment.

2.4 Cloud-Based Monitoring

Cloud-based services allow sensor data collected by IoT devices to be stored and accessed remotely. Platforms such as ThingSpeak provide facilities for collecting and visualizing sensor data through the Internet [16]. Blynk is another IoT platform that provides remote monitoring and visualization capabilities [17].

Instead of relying on a predefined IoT dashboard, the proposed system uses Google Sheets for cloud-based data logging and a custom web dashboard for visualization. A Python backend retrieves the stored data and provides it to the dashboard.

The data flow of the proposed system can be represented as:

ESP32 → Wi-Fi → Google Sheets → Python Backend → Web Dashboard

This architecture provides both cloud-based data storage and remote access to the monitored information.

2.5 Related Works

Several studies have explored the use of IoT technologies for environmental and air-quality monitoring. Previous work has demonstrated the use of sensors and wireless communication to collect and remotely monitor environmental data [19]. ESP32-based monitoring systems have also been investigated because of the microcontroller's integrated Wi-Fi capability and suitability for low-cost IoT applications [22]. Cloud-based platforms such as ThingSpeak have been used for storing and visualizing sensor data remotely [16], while Blynk has been used to provide IoT-based remote monitoring and visualization [17]. These studies demonstrate the feasibility of combining low-cost sensors, microcontrollers, wireless communication, and cloud services for environmental monitoring. The proposed system builds on these approaches by integrating multiple sensors with ESP32, cloud-based data logging, a Python backend, and a custom web dashboard.

2.6 Research Gap

Existing IoT-based air-quality monitoring systems have demonstrated the use of sensors, microcontrollers, wireless communication, and cloud platforms for collecting and monitoring environmental data. However, many existing approaches either focus on local monitoring or depend on predefined IoT platforms for data visualization. The proposed system addresses this gap by integrating multiple low-cost sensors with an ESP32, cloud-based data logging, a Python backend, and a custom web dashboard in a single monitoring system. In addition, the system provides a project-specific Air Quality Score and historical data visualization, allowing users to monitor sensor conditions remotely through a web interface.

2.7 Summary

The literature review shows that IoT technology, ESP32, low-cost gas sensors, and cloud-based services can be effectively combined for environmental monitoring. Previous studies provide the basis for using wireless microcontrollers and cloud platforms for remote monitoring. Based on these concepts, the proposed system integrates ESP32-based sensing, cloud data logging, backend processing, and a web dashboard for air-quality-related monitoring.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter presents the methodology used to design and develop the Cloud-Based IoT Air Quality Monitoring System. It describes the overall system architecture, development model, process flow, use case, air quality calculation method, system workflow, budget, and project schedule. The methodology integrates the ESP32 microcontroller with MQ2, MQ135, and DHT11 sensors for environmental data collection, followed by Wi-Fi-based data transmission to Google Sheets. The collected data is processed through a Python backend and presented on a custom web dashboard for remote monitoring and historical analysis.

3.2 Software Development Life Cycle (SDLC) Model

The project follows the **Waterfall Model** as the software development life cycle. The Waterfall Model follows a sequential development process in which each major phase is completed before proceeding to the next phase.

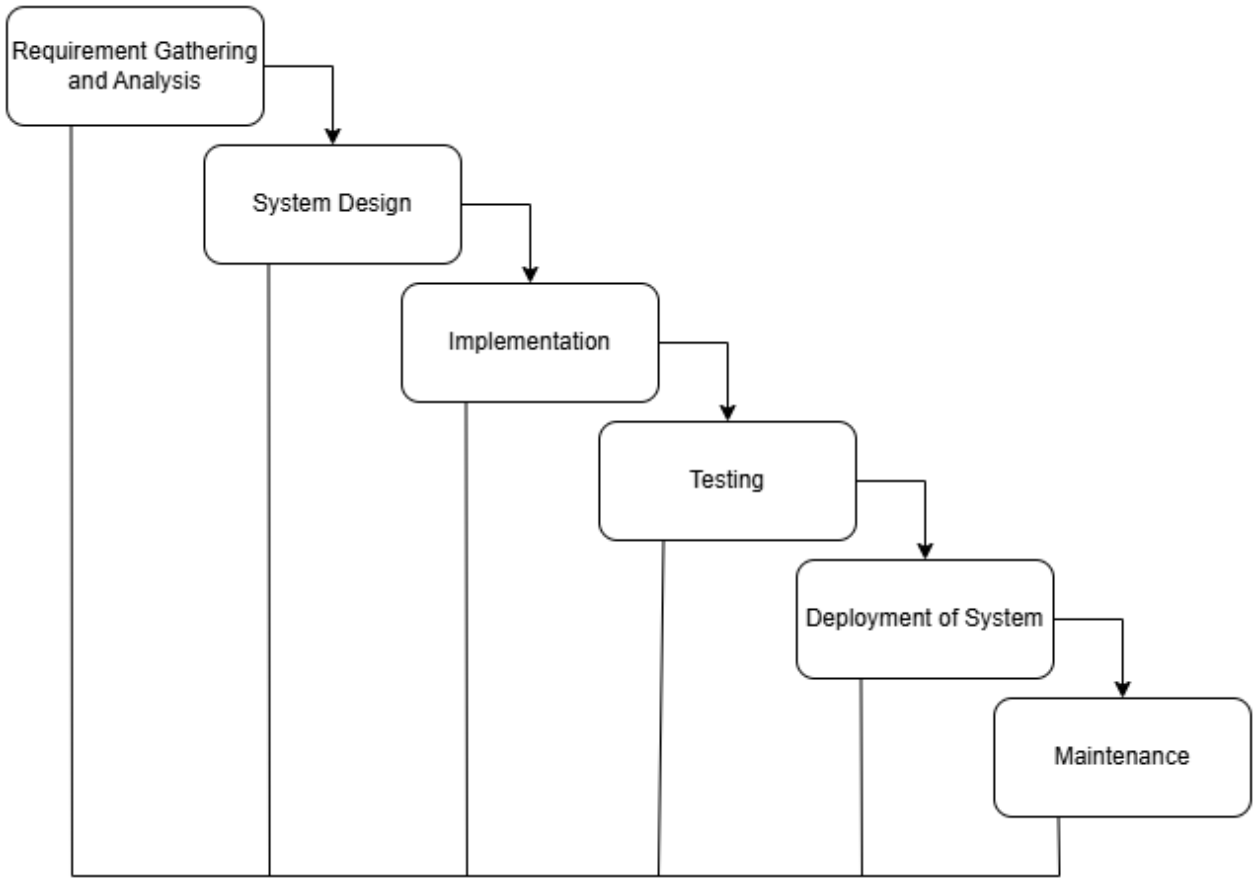


Figure 3.1: Waterfall Model (SDLC)

3.3 System Architecture

The system architecture is divided into four parts: Sensors, Microcontroller, Local Display, and Cloud Pipeline.

- **Sensors:** MQ2 and MQ135 gas sensors and the DHT11 temperature/humidity sensor collect environmental data.
- **Microcontroller (ESP32):** Reads all sensor inputs, drives the local LCD, and logs readings to Google Sheets over Wi-Fi.
- **Local Display:** A 16×2 LCD shows real-time readings on-site.
- **Cloud Pipeline:** A Python backend (live-data.py) reads logged data from Google Sheets and serves it to a custom web dashboard, deployed on Vercel, where readings are scored, classified, charted, and made available for historical review.

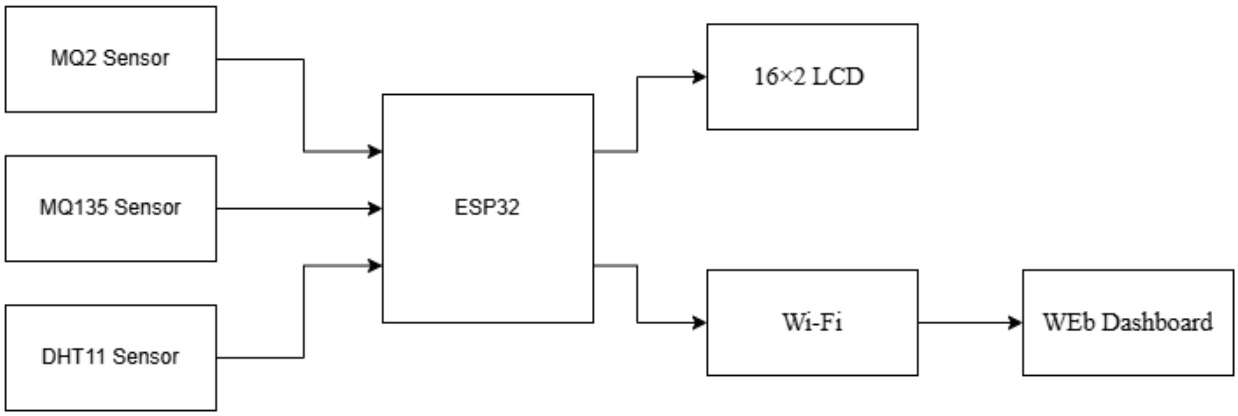


Figure 3.2: Block Diagram of AQMS

Description: This diagram illustrates the data flow where the MQ2, MQ135, and DHT11 sensors send readings into the ESP32, which updates the 16×2 LCD locally and logs data to Google Sheets. The Python backend then reads from the Sheet and serves the data to the web dashboard for remote viewing.

3.4 Use Case Diagram

The Use Case Diagram represents the interaction between the user and the system. The main actor is the User, who can:

- View real-time sensor readings on the local LCD
- View live sensor data and air quality status on the web dashboard
- View historical readings, filtered by date range or air quality status
- Export historical readings as a CSV file

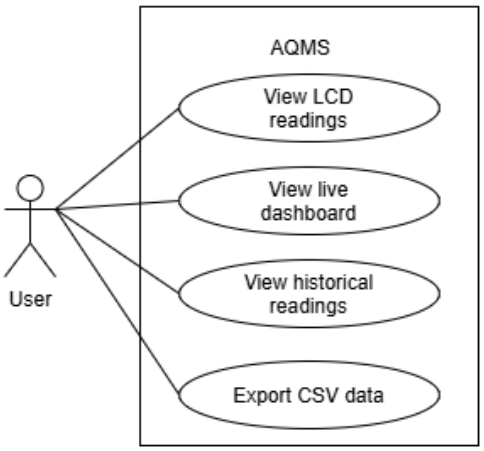


Figure 3.3: Use Case Diagram

3.5 Process Modelling Using Flowchart

The flowchart below represents the logical sequence of the air quality scoring process, from raw sensor input through to the classification displayed on the dashboard.

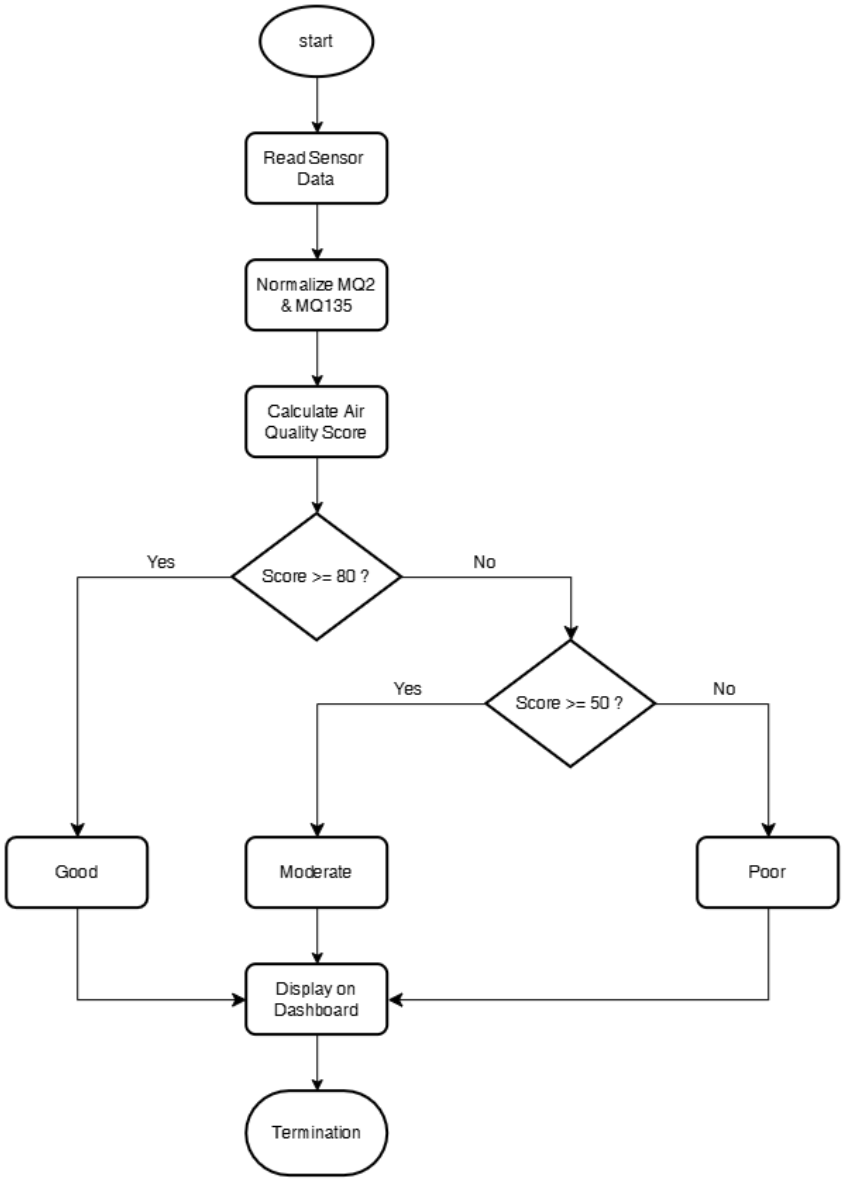


Figure 3.4: Air Quality Scoring Flowchart

The process begins when sensor data is read from the MQ2 and MQ135 sensors. These raw readings are normalized to a 0–100 pollution scale using the calibrated MIN/MAX ranges for each sensor. The normalized values are then combined into a weighted Pollution Index and converted into an overall Air Quality Score. The system evaluates the score against two thresholds: scores of 80 or above are classified as Good, scores between 50 and 79 as Moderate, and scores below 50 as Poor. The resulting classification is then displayed on the dashboard.

3.6 Air Quality Calculation

The system calculates a project-specific Air Quality Score using the calibrated MQ-2 and MQ-135 sensor readings obtained through the ESP32 ADC. Since the ESP32 uses a 12-bit ADC, the readings are normalized using the range of **0–4095**.

The normalized sensor values are calculated as:

$$MQ2_{norm} = (MQ2_{ADC} / 4095) \times 100$$

$$MQ135_{norm} = (MQ135_{ADC} / 4095) \times 100$$

The normalized values are then combined using predefined weights, with MQ-2 contributing 40% and MQ-135 contributing 60%:

$$PI=(0.4\times MQ2_{norm}) + (0.6\times MQ135_{norm})$$

The Air Quality Score is calculated as:

$$AQScore=100-PI$$

The resulting score is classified as:

AQ Score ≥ 80 → Good;

50 ≤ AQ Score < 80 → Moderate;

AQ Score < 50 → Poor

3.7 Budget Analysis

The estimated budget for the Air Quality Monitoring System is kept low as it is designed as a prototype project using affordable and easily available components. The cost breakdown of the required hardware components is given below:

Table 3.1: Budget Analysis

S.N.	Components	Quantity	Unit Price (Rs.)	Total Price (Rs.)
1	ESP32 Board	1	1000	1000
2	MQ2 Gas Sensor	1	300	300
3	MQ135 Gas Sensor	1	400	400
3	DHT11 Sensor	1	250	250
4	16×2 LCD Display	1	450	450
5	Jumper wires	30	5	150
6	Breadboard	1	120	120
7	Project Board	1	200	200
8	Buck Converter	1	200	200
9	Rechargeable Batteries	2	200	400
Total Estimated Cost (Rs.)				**3470**

3.8 Work Schedule

The work schedule for the Air Quality Monitoring System is planned over 12 weeks, dividing the project into clear phases, including requirement analysis, system design, implementation, testing, and documentation. This ensures systematic development and timely completion of the project.

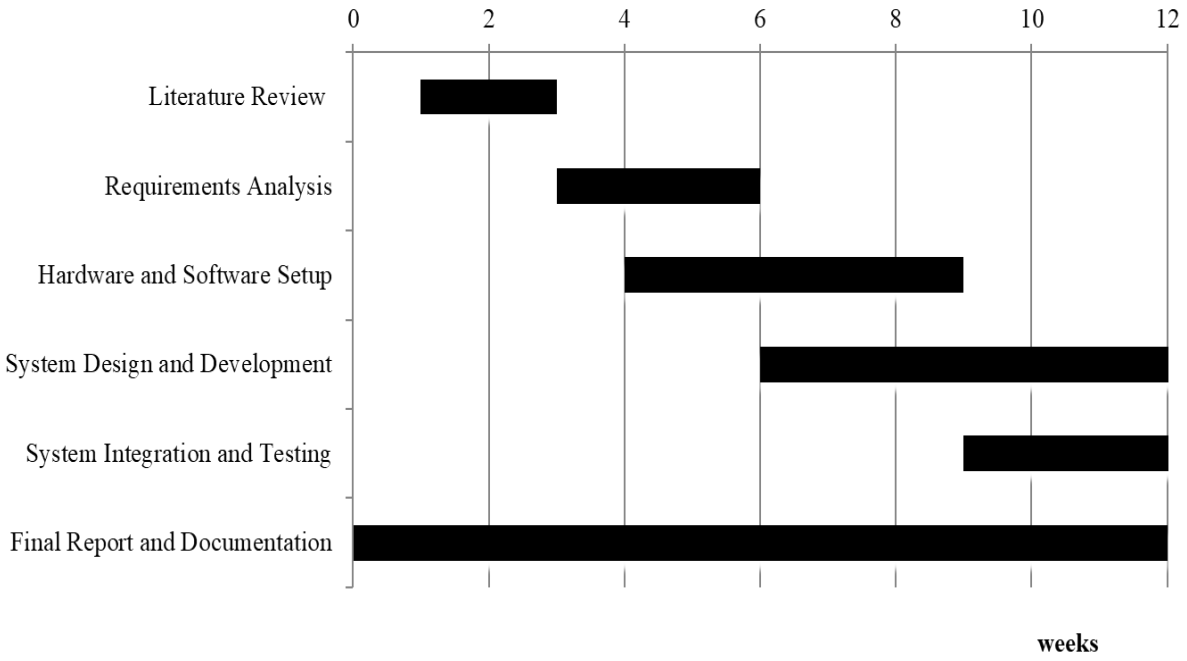


Figure 3.5: Gantt Chart

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Final Outcome

The final outcome of the project is a functional Cloud-Based IoT Air Quality Monitoring System integrating environmental sensing, wireless communication, cloud data storage, backend processing, and web-based monitoring. The completed system collects sensor readings using the MQ2, MQ135, and DHT11 sensors through the ESP32 and provides both local and remote access to the collected data.

The system successfully integrates the hardware prototype with Google Sheets, a Python backend, and a custom web dashboard. The implementation and testing of these components are presented in the following sections.

4.2 Hardware Implementation

The hardware implementation consists of the **ESP32, MQ2, MQ135, DHT11, and 16×2 LCD**. The ESP32 collects readings from the sensors, processes the data, and displays the readings on the LCD. The completed components were connected and tested to verify proper sensor data acquisition and display operation.

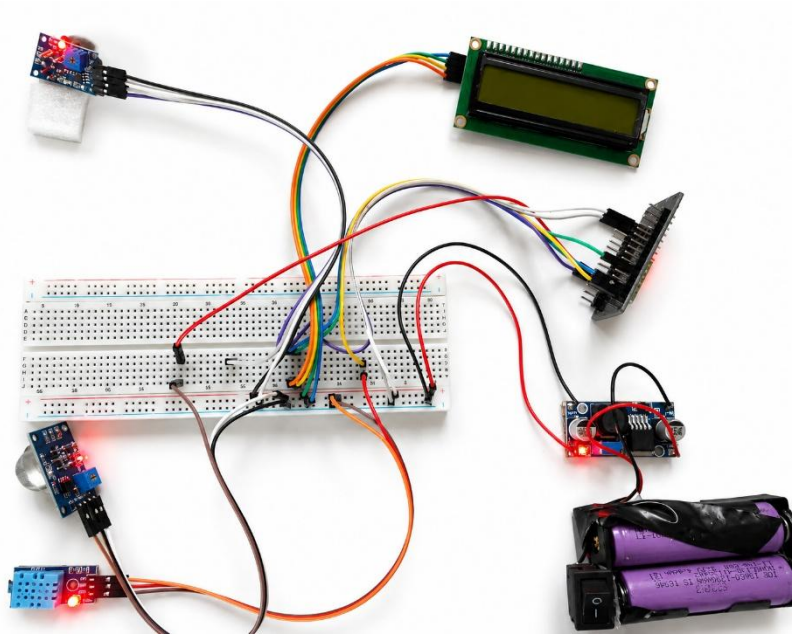


Figure 4.1: Completed Hardware Prototype

4.3 ESP32 Firmware Implementation

The ESP32 firmware was developed to collect readings from the MQ2, MQ135, and DHT11 sensors, display the data on the 16×2 LCD, and transmit the collected readings through Wi-Fi. The firmware also supports the data flow required for cloud logging and remote monitoring.

The program continuously reads the sensors and sends the updated values to the cloud system at regular intervals.

4.4 Cloud Data Logging

The collected sensor readings are transmitted from the ESP32 through Wi-Fi and stored in Google Sheets for cloud-based data logging. The stored data provides a record of sensor readings that can be accessed by the backend and used for dashboard visualization and historical monitoring.

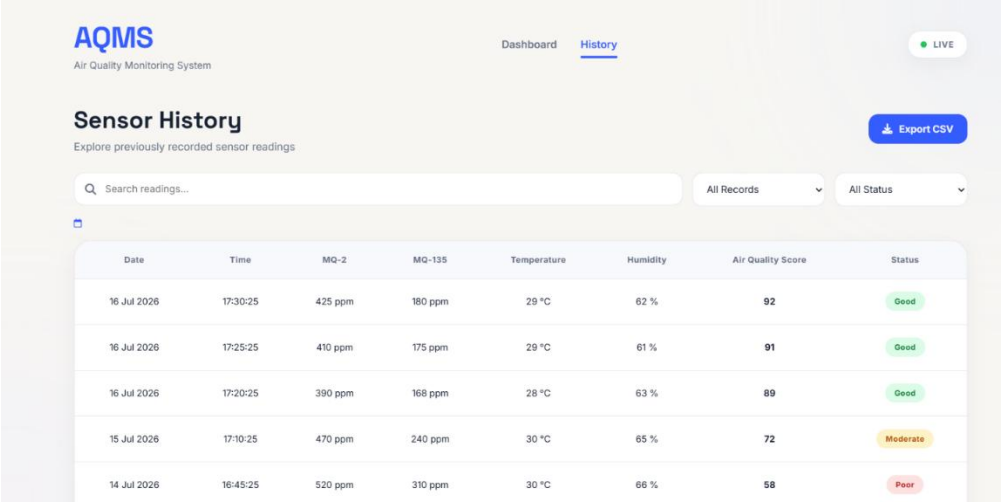


Figure 4.2: Sensor Data Logged in Google Sheets

4.5 Python Backend Implementation

The Python backend retrieves sensor data from Google Sheets and provides it to the web dashboard. It acts as an intermediary between cloud data storage and the frontend, enabling the display of current and historical sensor readings.

4.6 Web Dashboard Implementation

The web dashboard was developed to provide a simple interface for remotely monitoring the collected sensor data. It displays the MQ2, MQ135, and DHT11 readings, along with the Air Quality Score and air quality classification.

The dashboard also provides historical data visualization, allowing users to observe changes in sensor readings over time. The interface retrieves the data through the Python backend and presents it in an organized form for easier monitoring.

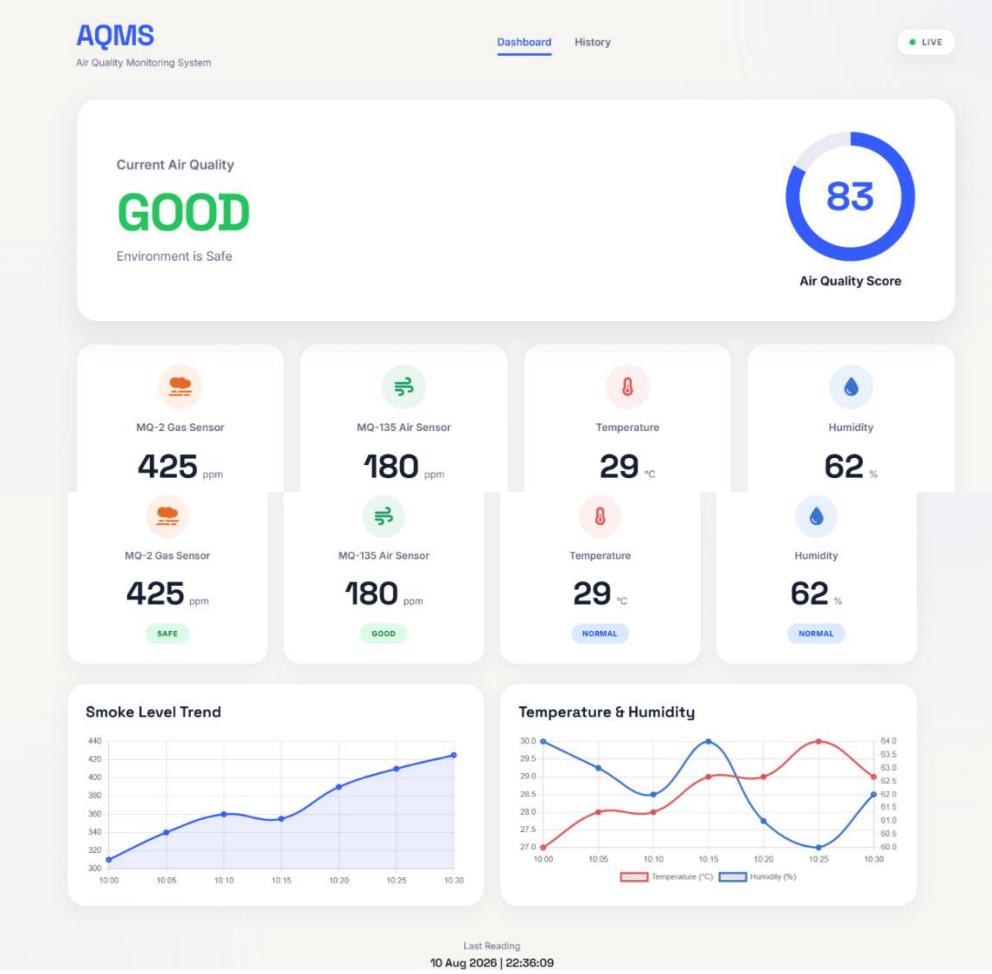


Figure 4.3: Implemented Web Dashboard

4.6 Testing

The developed system was tested to verify the proper operation of its major hardware, communication, cloud, and monitoring functions. Individual components and the complete integrated system were checked using practical test cases.

The following functions were tested:

- Sensor data acquisition from MQ2, MQ135, and DHT11.
- Display of sensor readings on the 16×2 LCD.
- Wi-Fi connectivity and data transmission.
- Storage of sensor data in Google Sheets.
- Retrieval and display of data on the web dashboard.
- Air Quality Score and classification.
- Continuous operation of the complete system.

4.7 Test Results

The testing of the developed system produced satisfactory results for the major functions of the proposed system. The sensors successfully provided readings, the ESP32 displayed the data on the LCD and transmitted it through Wi-Fi, and the data was stored in Google Sheets and displayed on the web dashboard.

Table 4.1: System Testing Results

S.N.	Test Case	Expected Results	Results
1	MQ2 sensor reading	Sensor reading is obtained	Pass
2	MQ135 sensor reading	Sensor reading is obtained	Pass
3	DHT11 reading	Temperature and humidity are obtained	Pass
4	LCD display	Sensor readings are displayed	Pass
5	Wi-Fi connection	ESP32 connects and transmits data	Pass
6	Cloud data logging	Data is stored in Google Sheets	Pass
7	Web dashboard	Data is displayed correctly	Pass
8	Air Quality Score	Score and classification are displayed	Pass

4.8 Discussion

The testing results show that the implemented system successfully performs the major functions of air quality monitoring. Sensor readings are collected through the ESP32, displayed locally on the LCD, transmitted through Wi-Fi, stored in Google Sheets, and presented on the web dashboard. The integration of hardware, cloud data logging, backend processing, and web monitoring demonstrates that the proposed system can provide both **local and remote monitoring** of environmental conditions. The test results also confirm that the major system functions operated as expected during testing.

Chapter 5
EPILOGUE

5.1 Conclusions

The Cloud-Based IoT Air Quality Monitoring System was successfully designed and implemented to provide real-time monitoring of environmental conditions. The system integrates MQ2, MQ135, and DHT11 sensors with an ESP32 to collect and process sensor readings.

The collected data is displayed locally on a 16×2 LCD and transmitted through Wi-Fi for cloud-based storage in Google Sheets. A Python backend and web dashboard provide remote access to the collected data, Air Quality Score, classification, and historical readings.

The implemented prototype demonstrates the practical integration of IoT hardware, cloud data logging, backend processing, and web-based monitoring for air quality observation. Testing confirmed that the major components and functions of the system operated as expected.

5.2 Limitations

The developed system has certain limitations that should be considered. The accuracy of the air quality assessment depends on the readings obtained from the MQ2 and MQ135 sensors and the conditions under which they operate. The Air Quality Score used in the system is a project-specific scoring method and should not be considered equivalent to a standardized government or professional AQI measurement.

The system also depends on a stable Wi-Fi connection for transmitting sensor data to the cloud and enabling remote monitoring. Since the prototype uses a limited number of gas and environmental sensors, it cannot measure all types of air pollutants. Furthermore, the system is intended for monitoring and awareness purposes rather than certified air-quality measurement.

5.3 Future Scope

The developed system provides a foundation for further improvement in IoT-based environmental monitoring. In future, the system can be extended by integrating additional sensors to monitor a wider range of pollutants and environmental parameters. The sensing unit can also be deployed at multiple locations to collect spatial air-quality data and provide a broader view of pollution conditions.

The cloud infrastructure can be enhanced to support long-term data storage,advanced visualization, and automated alerts based on user-defined thresholds. Machine learning techniques can also be incorporated to analyze historical sensor data and predict pollution trends. Further improvements in sensor accuracy, power management, and communication reliability can make the system more suitable for continuous outdoor deployment and larger-scale environmental monitoring.

REFERENCES

[1] ECIT Engineering, “IoT-Based Air Quality Monitoring System using ESP32,” YouTube, 2024.

[2] A. Sharma and R. Patel, “Comparative analysis of single-core microcontrollers and dual-core IoT SoC architecture for real-time applications,” 2019.

[3] S. Prasad *et al.*, “GSM-based hazardous gas leakage detection and alerts using 8-bit microcontrollers,” 2019.

[4] M. A. Khan and S. Mishra, “Environmental telemetry logging via ESP8266 Wi-Fi frameworks on cloud databases,” 2021.

[5] Blynk, “Blynk IoT documentation: Virtual datastreams, webhook automation controllers, and SMTP mail integration architectures,” 2022.

[6] J. Alphonse *et al.*, “Humidity and temperature compensation matrices for metal-oxide semiconductor (MOS) gas sensors,” 2022.

[7] L. Gomez and K. Lund, “Event-driven vs. polling network topologies in low-power wide-area IoT edge nodes.”

[8] Espressif Systems, *ESP32-WROOM-32 Datasheet and Technical Reference Manual*, 2020.

[9] K. Radhakrishnan *et al.*, “Edge computing paradigms in low-cost air quality sensor networks: Minimizing cloud dependency,” 2023.

[10] W. W. Royce, “Managing the development of large software systems,” in *Proc. IEEE WESCON*, 1970, pp. 1–9.

[11] Google, “Air quality monitoring system,” Google Web Resource, 2025.

[12] Google, “IoT environmental monitoring,” Google Web Resource, 2025.

[13] Components101, “MQ-2 gas sensor,” Components101.

[14] Components101, “MQ-135 gas sensor,” Components101.

[15] Adafruit, “DHT11 basic temperature-humidity sensor,” *Adafruit Learning System*.

[16] MathWorks, “ThingSpeak IoT analytics,” MathWorks. ThingSpeak is described by MathWorks as a cloud IoT analytics platform for collecting, visualizing, and analyzing live data streams.

[17] World Health Organization, *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva, Switzerland: World Health Organization, 2021.

[18] Hackatronic, “Air Quality Monitoring System Project with ESP32-C6 and Blynk IoT,” 2024.

[19] *Monitoring and Management Using Sensors*. Elsevier, 2025.

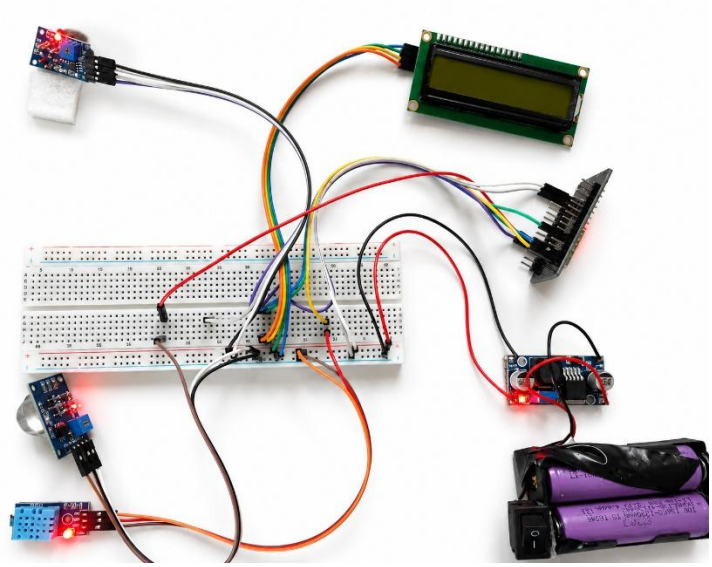
[20] S. Kumar, P. Verma, and R. Singh, “Real-time environmental monitoring using IoT and cloud computing,” *International Journal of Scientific Research in Computer Science Engineering and Information Technology*, vol. 7, no. 3, pp. 112–118, 2022.

[21] SparkFun Electronics, “How to use a breadboard,” *SparkFun Tutorials*, 2024.

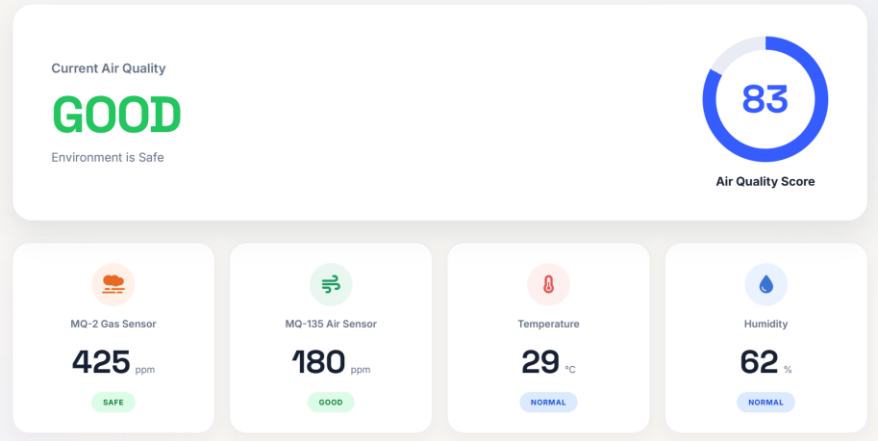
[22] Electronics Tutorials, “Gas sensors and air quality monitoring basics,” 2024.

[23] A. Sharma and R. Patel, “IoT based air quality monitoring system using ESP32,” *International Journal of Engineering Research & Technology (IJERT)*, vol. 10, no. 5, pp. 234–238, 2021.

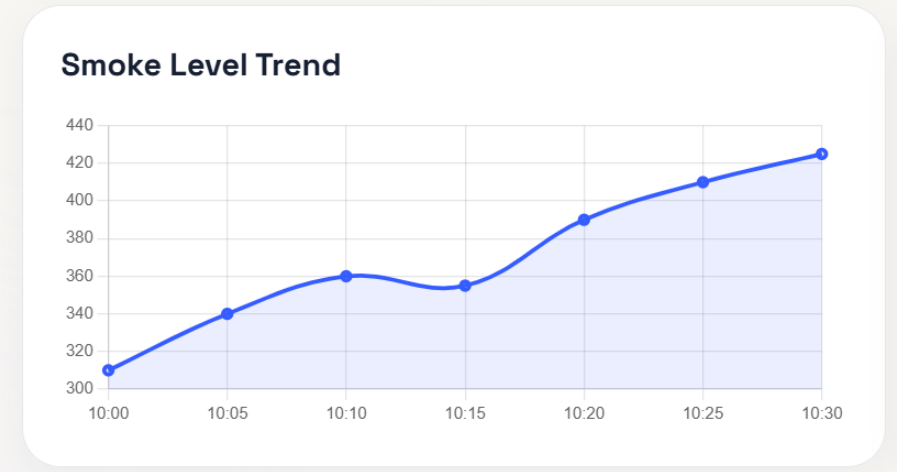
APPENDICES



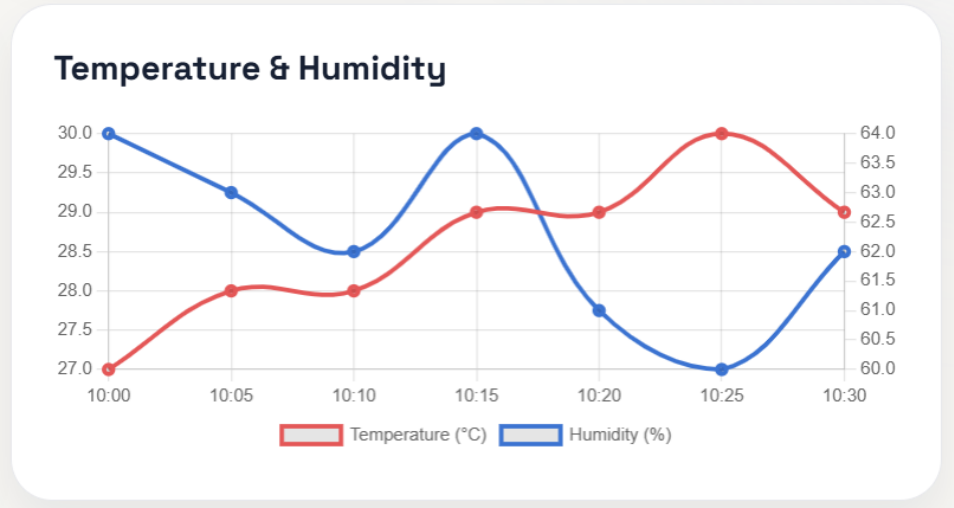
Annex 1: Hardware Integration



Annex 2: Dashboard



Annex 3: Smoke Level Trend



Annex 4: Temperature & Humidity Graph

Date	Time	MQ-2	MQ-135	Temperature	Humidity	Air Quality Score	Status
16 Jul 2026	17:30:25	425 ppm	180 ppm	29 °C	62 %	92	Good
16 Jul 2026	17:25:25	410 ppm	175 ppm	29 °C	61 %	91	Good
16 Jul 2026	17:20:25	390 ppm	168 ppm	28 °C	63 %	89	Good
15 Jul 2026	17:10:25	470 ppm	240 ppm	30 °C	65 %	72	Moderate
14 Jul 2026	16:45:25	520 ppm	310 ppm	30 °C	66 %	58	Poor
12 Jul 2026	15:20:25	350 ppm	155 ppm	28 °C	60 %	88	Good
10 Jul 2026	14:10:25	430 ppm	210 ppm	29 °C	64 %	78	Moderate
20 Jun 2026	12:30:25	320 ppm	140 ppm	27 °C	58 %	90	Good
15 May 2026	11:45:25	480 ppm	260 ppm	31 °C	68 %	68	Moderate
10 Dec 2025	10:15:25	550 ppm	350 ppm	24 °C	55 %	52	Poor

Annex 5: History Table